

TOULOUSE INP N7

Le Vibe Coding

Développer avec l'IA : Guide pratique et retours d'expérience

TONTON STUDIO
THE FUTURE IS BACK

Qu'est-ce que le vibe coding ?

Le vibe coding représente une nouvelle approche du développement logiciel où l'intelligence artificielle devient un partenaire actif dans le processus de création. Au lieu de coder ligne par ligne, **vous collaborez avec des modèles de langage** pour conceptualiser, implémenter et itérer rapidement sur vos idées.

Cette méthode transforme la façon dont nous abordons les problèmes techniques : l'IA suggère des solutions, génère du code, identifie des bugs, et vous permet de vous concentrer sur l'architecture et la vision globale.

Mais attention : à l'heure actuelle, il est fortement déconseillé de se lancer dans cette approche pour développer un produit fini sans un **minimum d'expertise technique** et un **maximum de supervision !**





L'évolution du vibe coding

1 Vibe coding "sauvage"

Multiples itérations avec un LLM : prompt→ bout de code → copiés-collés → prompt….etc. Essais-erreurs sans méthode, gestion du contexte "à la mano", génération de code sans compréhension ni validation.

2 Phase d'apprentissage

Découverte des outils IA spécialisés, premiers prompts structurés, début de méthodologie mais processus encore chaotique.

3 Vibe coding professionnel

Workflows établis, documentation systématique, gestion rigoureuse du contexte, intégration complète dans les pratiques DevOps.

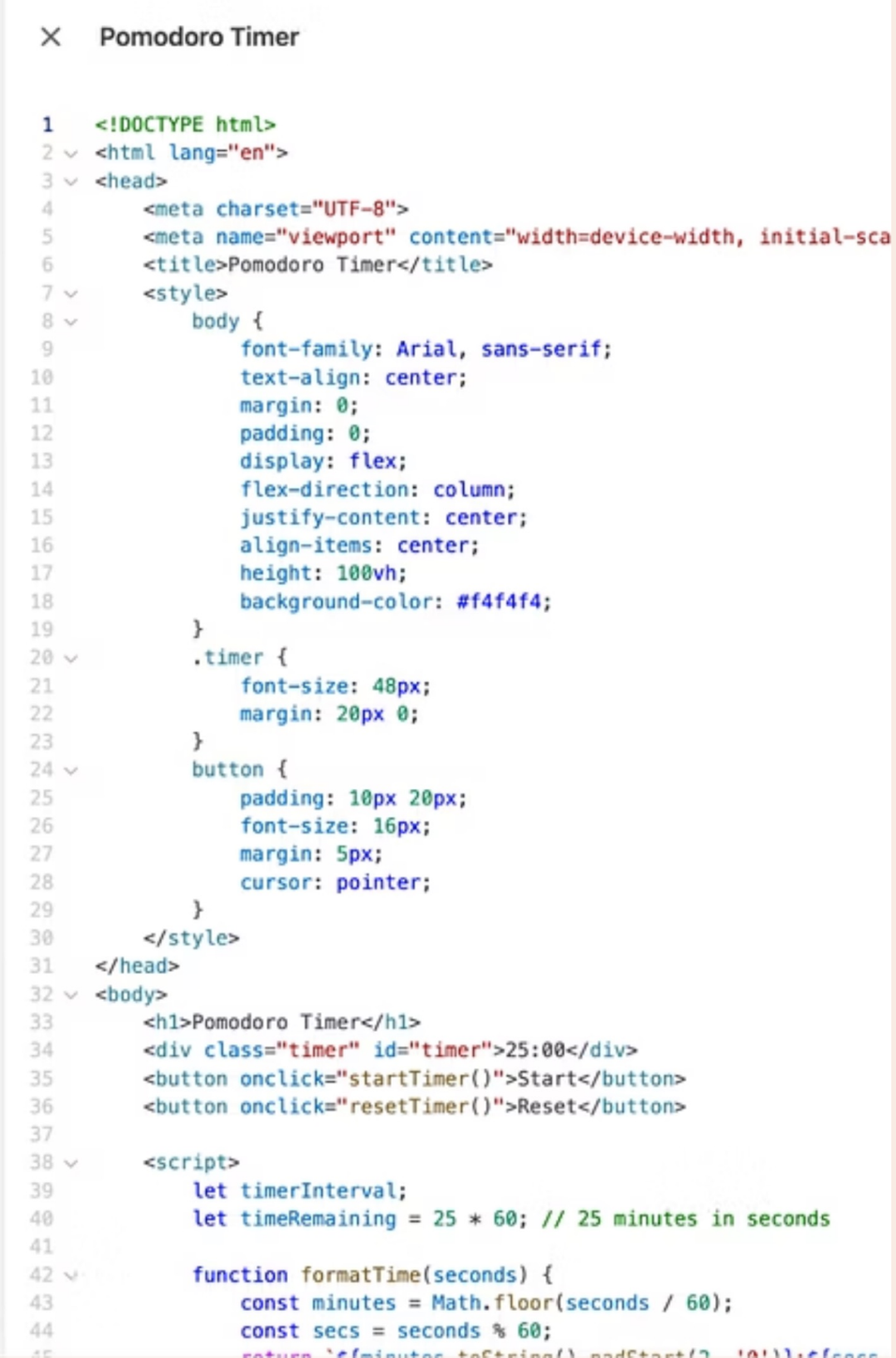
Mon parcours vibe coding : du chaos à la méthode

Le **vibe coding**, c'est l'art de laisser **l'intuition** guider le développement de code, souvent en collaboration étroite avec des outils d'IA. C'est une danse entre la **créativité** et la **logique**, où l'on "ressent" la bonne direction à prendre, plutôt que de suivre un plan rigide.



Étape 1 : les premiers pas avec ChatGPT

Mes débuts ont été marqués par l'expérimentation. Des prompts simples, beaucoup de copier-coller, surtout pour des projets web. C'était excitant, mais le processus était loin d'être structuré.



Étape 2 : l'arrivée de ChatGPT 4o et Canvas

L'intégration de ChatGPT 4o et l'utilisation de Canvas ont marqué un tournant. J'ai commencé à rédiger des **CDC techniques**, à réfléchir à **l'architecture et l'arborescence du projet** et à faire des **sauvegardes régulières**. Une approche plus organisée commençait à émerger.



Étape 3 : la découverte de Lovable et Windsurf

Ces outils ont d'abord semblé magiques, promettant une fluidité incroyable. J'ai été séduit par leur potential avec une fenêtre contextuelle paraissant extrêmement bien gérée… Mais j'ai rapidement rencontré des limitations (notamment techniques et financières) qui m'ont poussé à chercher d'autres solutions.



Étape 4 : Claude Desktop et Model Context Preservation

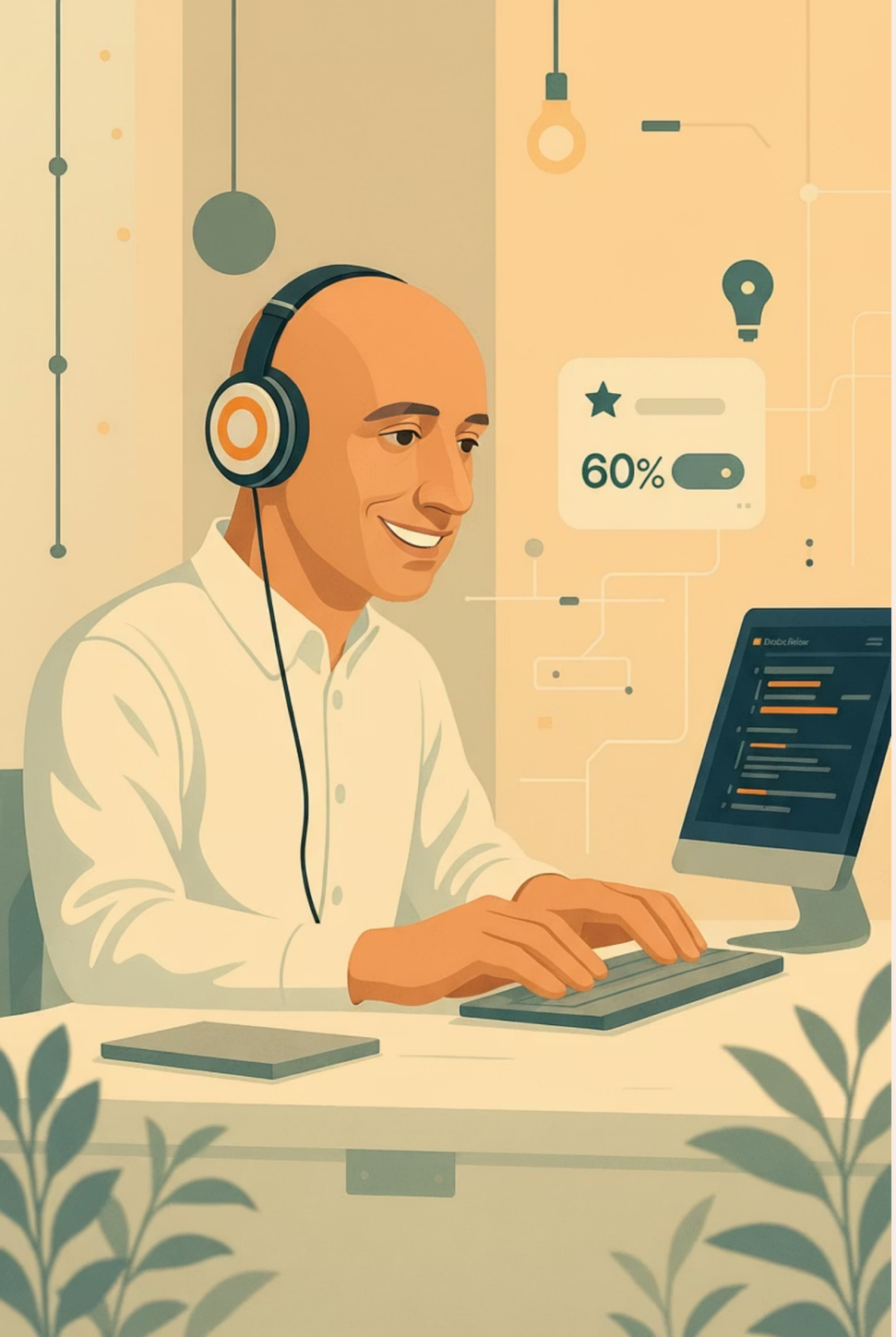
Avec Claude Desktop et les **MCP** (Model Context Protocol), j’ai commence à mettre en place un workflow beaucoup plus performant. La gestion du contexte restait laborieuse, mais les résultats étaient significativement meilleurs.



Étape 5 : l'ère de Claude Code

Claude Code a apporté une fluidité et une mémoire contextuelle que je n'avais pas connues auparavant, notamment grâce à son **autocompression du contexte**, ses possibilités agentiques, sa compatibilité MCP et ses customs commands.

Cela a transformé ma façon de coder avec l’IA !



Étape 6 : vers l'optimisation continue du workflow

Aujourd'hui, mon workflow est bien plus affiné, intégrant des sous-agents spécialisés, des outils optimisant toujours plus mon workflow et une automatisation croissante. Je suis dans une démarche d'amélioration continue, cherchant toujours à booster l'efficacité et la qualité de mon vibe coding.

Retour d'expérience personnel

⚠️ **Vigilance santé mentale** : Les LLM sont extrêmement convaincants dans leurs réponses, même quand ils se trompent. Cette confiance artificielle peut vous pousser à douter de votre propre jugement.

Le piège de la productivité

Quand l'IA accélère votre travail, il devient tentant de continuer "encore un peu" après avoir terminé une grosse fonctionnalité. **Résistez à cette tentation.**

L'importance des pauses

En milieu professionnel, arrêtez-vous en fin de journée même s'il vous reste des tokens. La fatigue mentale s'accumule plus vite qu'avec du code traditionnel.

Célébrer les victoires

Après avoir résolu un gros problème, prenez le temps de savourer cette réussite. Ne vous précipitez pas sur la tâche suivante immédiatement.

Outils populaires pour le vibe coding

GitHub Copilot

Avantages : Intégration IDE native, suggestions contextuelles rapides, excellent pour l'autocomplétion.

Inconvénients : Limité aux suggestions courtes, parfois répétitif, nécessite une connexion constante.

Codex (OpenAI)

Avantages : Intégration dans divers environnements (cloud, GitHub, local via VS Code, Windsurf ou Cursor), suggestions contextuelles intelligentes, polyvalence sur tous types de problèmes de code.

Inconvénients : Dépendant de l'environnement d'intégration, parfois limité par la fenêtre de contexte, nécessite une configuration selon l'outil utilisé.

Claude Code

Avantages : Environnement de développement intégré avec IA, édition directe du code, excellent pour l'architecture et le prototypage rapide, autocompression du contexte.

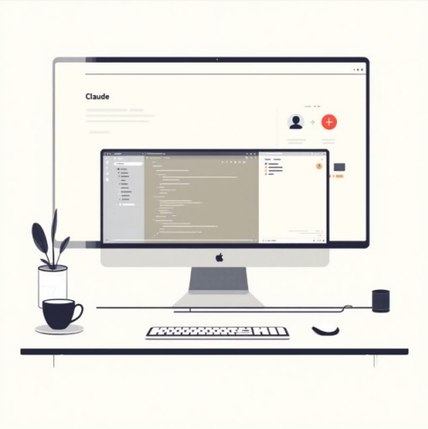
Inconvénients : Coût plus élevé, disponibilité parfois limitée selon les régions, moins adapté aux très gros projets.

Cursor / Windsurf

Avantages : Éditeurs complets avec IA intégrée, édition multi-fichiers, workflows fluides.

Inconvénients : Nouveaux outils, écosystème en développement, courbe d'apprentissage.

Comparatif détaillé : Claude Desktop + MCP vs Claude Code



Claude Desktop + MCP

Avantages clés :

- **Intégration avancée :** Connexion profonde avec les outils externes et votre environnement de développement.
- **Contrôle granulaire :** Maîtrise complète sur la configuration, idéal pour les pipelines CI/CD.
- **Gestion de projets complexes :** Facilite l'orchestration de systèmes à grande échelle.

À considérer :

- Configuration initiale plus complexe.
- Courbe d'apprentissage potentiellement plus raide.

Utilisation optimale : Projets d'entreprise, intégration de systèmes existants, refactoring à grande échelle.



Claude Code

Avantages clés :

- **Expérience intégrée :** Environnement de développement complet avec IA pour la génération et la modification de code.
- **Édition directe :** Modifiez le code sans copier-coller manuel.
- **Prototypage rapide :** Idéal pour des cycles de développement courts et l'expérimentation.

À considérer :

- Moins de flexibilité pour les workflows très personnalisés.
- Peut être limité sur des bases de code très vastes ou architectures distribuées.

Utilisation optimale : Développement personnel, projets open source, apprentissage, prototypage rapide d'applications.

Claude Desktop + MCP : l'orchestrateur puissant

Avantages pour le développement :

- **Model Context Protocol (MCP) avancé :** connexion profonde et étendue aux outils externes (système de fichiers, bases de données, APIs, services cloud). Permet à Claude d'interagir directement avec votre environnement de développement.
- **Contrôle total :** maîtrise complète sur la configuration de l'environnement, les permissions d'accès et la gestion des dépendances. Idéal pour les pipelines CI/CD complexes.
- **Orchestration complexe :** facilite la gestion de projets de grande envergure nécessitant des interactions sophistiquées entre plusieurs systèmes et outils.
- **Débugging puissant :** possibilité d'inspecter et de modifier le code et l'environnement en temps réel via Claude, offrant une expérience de débugging assistée par IA.

Inconvénients :

- **Configuration initiale :** nécessite une configuration plus complexe et une gestion du contexte manuelle.
- **Courbe d'apprentissage :** peut être plus intimidant pour les développeurs moins expérimentés ou ceux qui débutent avec l'IA en développement.

Pourquoi l'utiliser :

Principalement pour son interface, et son canvas permettant de faire des petits prototypes très rapidement.

Malheureusement sa fenêtre contextuelle est trop limité.

Il est tout de même très intéressant pour commencer : en effet, ses limitations contextuelles m’ont permis de mieux comprendre comment mettre en place un workflow optimisé.

Claude Code : l'IDE intégré nouvelle génération

Avantages pour le développement :

- **Expérience intégrée :** un environnement de développement (IDE) complet avec des capacités d'IA intégrées, optimisé pour la génération et la modification de code (le CLI c’est la vie).
- **Édition directe :** Possibilité d'éditer le code directement dans l'interface, sans copier-coller manuel.
- **Prototypage rapide :** idéal pour des cycles de développement rapides, la création de POC (Proof of Concept) ou l'expérimentation de nouvelles idées sur n’importe quelle stack.
- **Accès aux MCP :** bénéficie également des capacités du Model Context Protocol, permettant d'interagir avec des fichiers et des outils, bien que l'intégration soit plus simplifiée que dans Claude Desktop.
- **Mémoire contextuelle dynamique :** système CLAUDE.md et autocompression intelligente du contexte pour maintenir la cohérence sur de longues sessions.

Inconvénients :

- **Moins de flexibilité :** moins adapté aux workflows de développement hautement personnalisés ou aux exigences de sécurité strictes des environnements d'entreprise (peut rapidement faire d’énormes boulettes).
- **Limitations d'échelle :** peut atteindre ses limites sur des projets avec une base de code très vaste ou des architectures distribuées complexes (comme tous les outils actuels de développement assistés par IA).

Quand l'utiliser :

Pour le développement personnel, les projets open source, l'apprentissage de nouveaux langages/frameworks, et le prototypage rapide d'applications web ou mobiles.

Alternatives gratuites à Claude Code

OpenCode (terminal) (→ lien)

Agent "à la Claude", open-source, Linux/Windows ; lit le projet et propose des modifs multi-fichiers ; nécessite une clé API ou un modèle local.

Gemini CLI (terminal) (→ lien)

Agent gratuit (en preview) performant sur gros contextes ; peut planifier, coder, tester ; nécessite un compte Google (cloud).

Continue (VS Code) (→ lien)

Panneau de chat dans l'éditeur ; modèle au choix (cloud ou local) ; réécritures ciblées dans les fichiers.

En secours : Windsurf (→ lien) / Amazon Q Developer (→ lien)

Pour l'auto-complétion quand un "agent" complet n'est pas disponible.

Reproduire l'expérience Claude Code sans licence (exemples)

1

Recette Opencode

Installer → Lancer une session → Dialoguer comme avec Claude Code via les modèles proposes → L'outil propose des changements : contrôler et committer.

2

Recette Continue + VS Code

Installer → Configurer un modèle custom (via Ollama + OpenRouter) → Demander des réécritures ciblées dans les fichiers.

3

Recette Gemini CLI

Installer → Se connecter → Décrire le résultat attendu → L'agent planifie, édite, teste et propose la suite.

4

Astuces Communes

Tenir un JOURNAL.md (prompts clés, décisions) ; faire des commits courts ; tester à chaque étape.

Liens utiles

- Aider
- OpenRouter
- Ollama

MCP : brancher des outils quand c'est utile



Ce que c'est

standard ouvert d'Anthropic pour relier un assistant IA à des outils, données et workflows via une architecture client-serveur ; idée clé : "brancher une fois, utiliser partout".



Dans ces options

OpenCode (ajout de serveurs MCP locaux/distants), Continue (déclaration dans .continue/mcpServers), Gemini CLI (compatible MCP, ex. via FastMCP).



Quand l'utiliser

- donner accès à une doc interne sans copier-coller,
- déclencher un script de build/QA,
- interroger une petite API ;

Non obligatoire pour le TP. 🙂

https://smithery.ai/

Pour aller plus loin :

- Documentation MCP officielle : https://modelcontextprotocol.io/
- FastMCP (pour faire ses propres MCP) : https://github.com/jlowin/fastmcp
- Exemples de serveurs MCP : https://github.com/modelcontextprotocol/servers
- L’incontournable Smithery : https://smithery.ai/

Best Practices : planification et process

01

Planification initiale

Définissez clairement vos objectifs, contraintes techniques et critères de succès avant de solliciter l'IA. Un cahier des charges précis = résultats de qualité.

Et maintenez le CDC et les docs techniques à jour en permanence (peut-être fait de manière automatique sur Claude Code grâce à CLAUDE.md !)

02

Prompts réutilisables

Créez une bibliothèque de prompts testés pour les tâches récurrentes : génération de tests, refactoring, documentation, debugging. Gagnez du temps et de la cohérence.Et même mieux : **faites des customs commands !**

03

Gestion du contexte

Maintenez une "memory bank" avec les décisions architecturales, conventions de code et contraintes projet. Injectez ce contexte dans chaque session pour des réponses pertinentes. Et malgré la capacité d'auto-compression automatique du contexte de Claude Code, **favorisez les nouvelles sessions fraîches** quand une tâche s'avère trop lourde !

04

Documentation continue

Journalisez systématiquement : CDC initial, bugs complexes résolus, grosses features implémentées. Cette documentation devient votre référence et votre contexte de session en session.

Best Practices : sécurité et qualité



Backups multiples

Système à trois niveaux : local (copies régulières), Git (commits fréquents), Time Machine ou équivalent. Vérifiez régulièrement que vos backups fonctionnent.



Anti-régression

Utilisez des checksums ou tests automatisés pour détecter les régressions introduites par l'IA. **Ne faites jamais confiance aveuglément aux modifications proposées !!!**



Sollicitations externes

Pour les décisions critiques, demandez une seconde opinion à une autre IA ou un sous-agent spécialisé. La validation croisée révèle souvent des angles morts.



💡 **Astuce pro :** Créez des sous-agents spécialisés (sécurité, performance, accessibilité) pour auditer le code généré par votre agent principal.

Pièges à éviter absolument

Le syndrome du "ça compile donc c'est bon"

L'IA génère du code syntaxiquement correct mais logiquement défaillant. Testez systématiquement, ne vous fiez pas aux apparences.

La perte de compréhension

Quand vous ne comprenez plus votre propre code, vous perdez le contrôle. Exigez des explications, documentez, simplifiez.

L'over-engineering assisté par IA

Les LLM adorent suggérer des solutions complexes. Gardez le principe KISS en tête : la simplicité est une vertu.

Négliger la sécurité

L'IA peut introduire des vulnérabilités subtiles (injection SQL, XSS, fuites de données). Auditez toujours le code sensible.

Ignorer les licences

Le code suggéré peut être inspiré de sources protégées. Vérifiez les licences, surtout pour du code commercial.

Tests réguliers et bissect Git

Les "régressions surprises" sont légion en vibe-coding : l'IA peut introduire des bugs subtils dans des parties du code apparemment non liées. Testez régulièrement l'ensemble de votre solution, pas seulement la partie modifiée. Utilisez la méthode du bissect Git pour identifier rapidement le commit responsable d'une régression - c'est un gain de temps énorme quand tout semblait fonctionner parfaitement la veille !

Veille technologique et synthèse

Comment faire sa veille efficacement

- **Newsletters ciblées :** TLDR, Hacker News Digest, AI Weekly
- **Twitter/X :** suivez les créateurs d'outils IA et les early adopters
- **Discord/Slack/Whatsapp :** communautés actives (Claude, OpenAI, local dev)
- **GitHub Trending :** découvrez les nouveaux outils émergents
- **Expérimentez :** testez un nouvel outil chaque mois



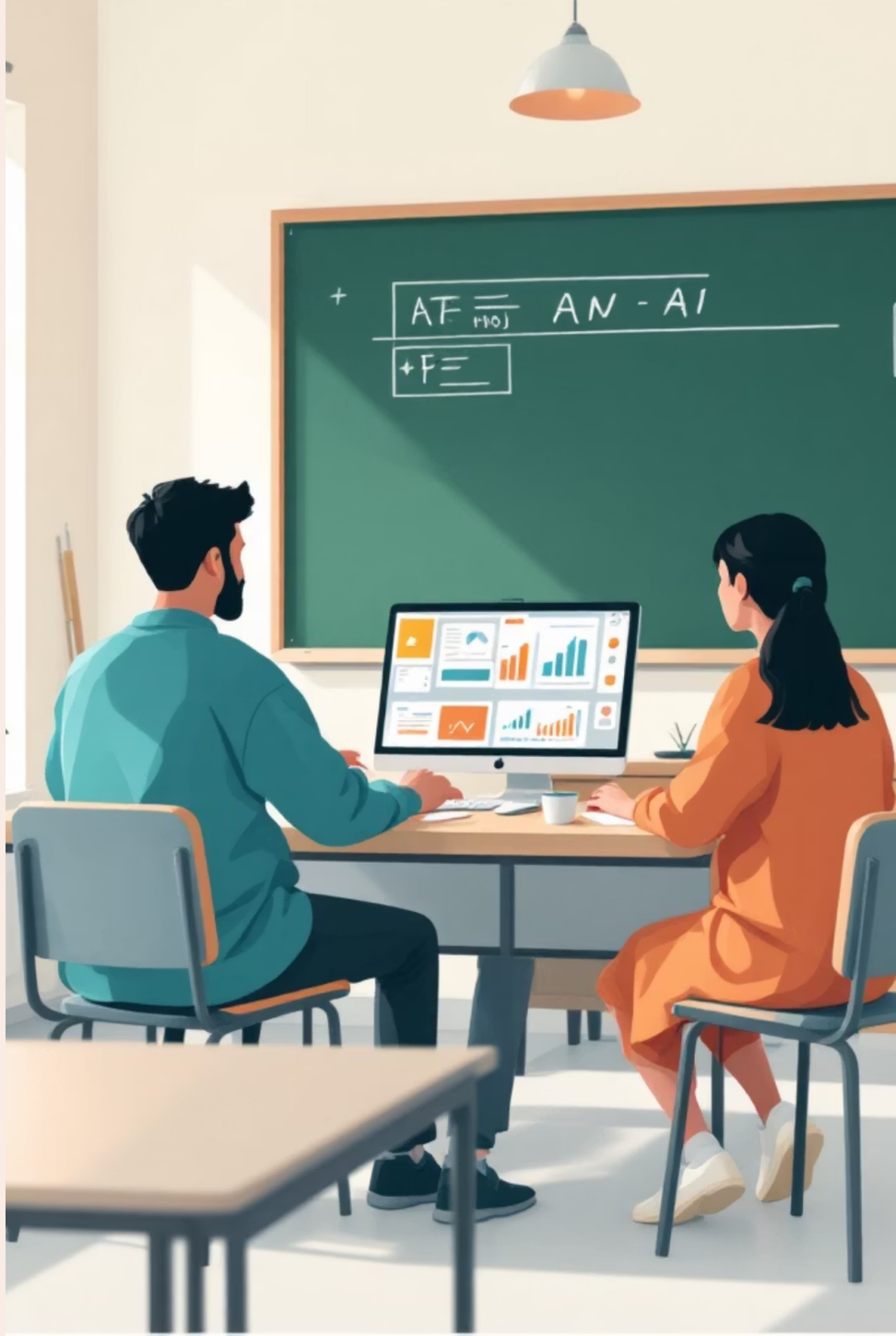
Points clés à retenir

Le vibe coding est un multiplicateur de productivité puissant qui nécessite discipline et méthode. Planifiez, documentez, sauvegardez. Protégez votre santé mentale en établissant des limites claires. Restez critique face aux suggestions de l'IA et maintenez votre compréhension du code.

À vous de jouer !

Après avoir exploré les stratégies et les bonnes pratiques, il est temps de **passer à l'action**. Mettez en application les concepts appris et **créez votre propre projet avec l'IA**.

C'est l'occasion d'expérimenter, de prototyper et de **transformer la théorie en résultats concrets !**



TP : créer votre application mobile avec l'IA

Objectif du projet :

Développer une application mobile innovante en utilisant les techniques de vibe coding apprises. Votre application doit exploiter au minimum 3-4 capteurs ou fonctionnalités natives de l'appareil mobile.

Capteurs et fonctionnalités à intégrer (choisir au moins 3-4) :

- **Caméra :** Capture photo/vidéo, reconnaissance d'objets, réalité augmentée
- **Microphone :** Enregistrement audio, reconnaissance vocale, analyse sonore
- **Accéléromètre :** Détection de mouvement, compteur de pas, jeux basés sur l'inclinaison
- **Gyroscope :** Orientation 3D, stabilisation, contrôles gestuels
- **GPS/Géolocalisation :** Cartographie, géofencing, tracking de parcours
- **Capteur de luminosité :** Adaptation automatique de l'interface
- **Capteur de proximité :** Interactions sans contact
- **Boussole :** Navigation, réalité augmentée géolocalisée

TP : Évaluation et Livrables

Critères d'évaluation

1	**Workflow personnel documenté (20%)** Votre méthodologie de vibe coding, outils utilisés, process de développement
2	**Innovation technique (25%)** Utilisation créative des capteurs, originalité de l'application
3	**Qualité du code (20%)** Architecture, lisibilité, bonnes pratiques
4	**Tests et déploiement (20%)** Tests sur device réel, compilation réussie, fonctionnalités opérationnelles
5	**Documentation (15%)** README, guide d'utilisation, retour d'expérience

Livrables attendus

- Code source complet avec historique Git
- Application compilée et testée sur mobile
- Documentation de votre workflow vibe coding + exemples de prompts clés
- Démonstration fonctionnelle (5-10 minutes)

Conseils :

Commencez par définir votre concept, puis établissez votre workflow de développement avec l'IA. Testez régulièrement sur device réel. N'hésitez pas à itérer et à documenter vos découvertes !